OS_Lab8_Experimental report

湖南大学信息科学与工程学院

计科 210X 甘晴void （学号 202108010XXX）

实验目的

通过完成本次实验，希望能达到以下目标

- 了解基本的文件系统系统调用的实现方法；
- 了解一个基于索引节点组织方式的Simple FS文件系统的设计与实现；
- 了解文件系统抽象层-VFS的设计与实现；

实验内容

实验七完成了在内核中的同步互斥实验。本次实验涉及的是文件系统，通过分析了解ucore文件系统的总体架构设计，完善读写文件操作，从新实现基于文件系统的执行程序机制（即改写do_execve），从而可以完成执行存储在磁盘上的文件和实现文件读写等功能。

练习**0**：填写已有实验

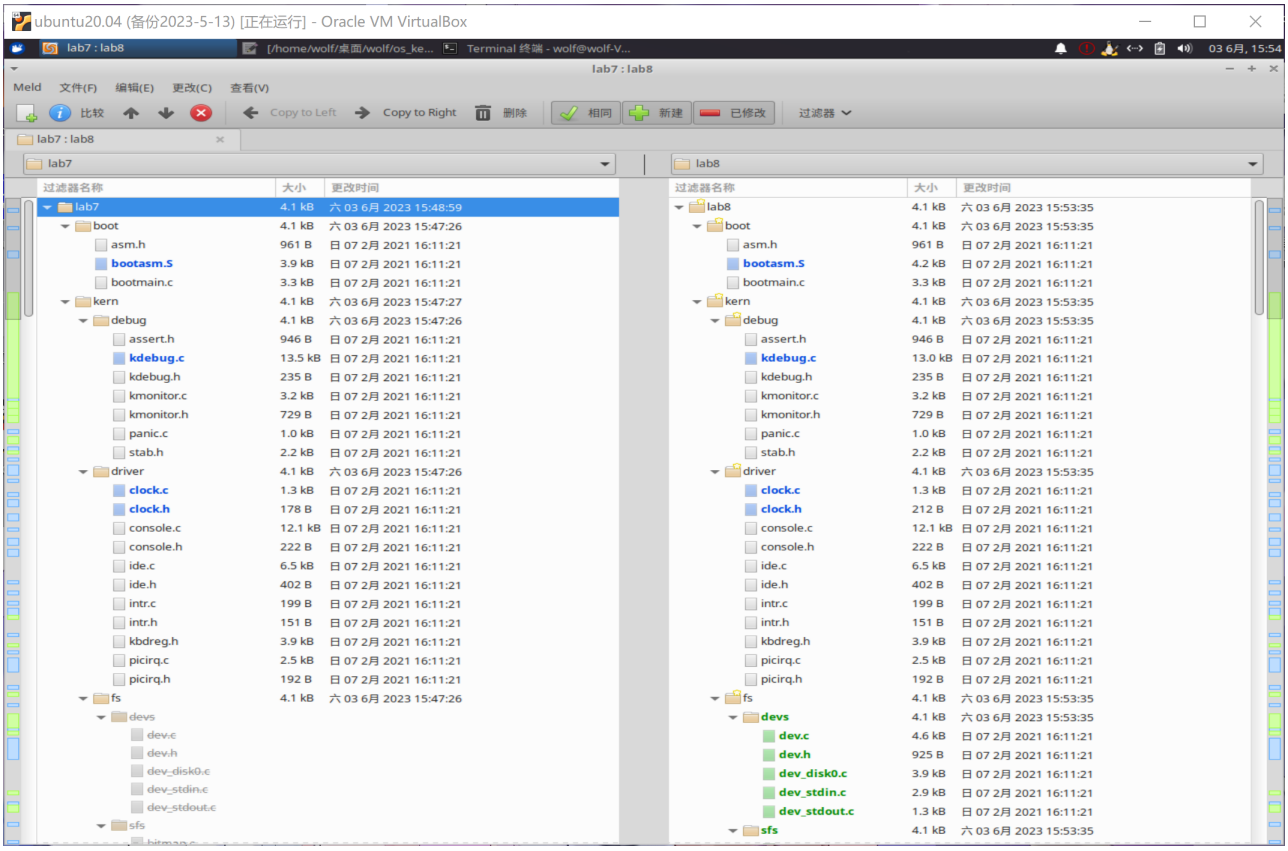
> 本实验依赖实验*1/2/3/4/5/6/7*。请把你做的实验*1/2/3/4/5/6/7*的代码填入本实验中代码中有*“LAB1”/“LAB2”/“LAB3”/“LAB4”/“LAB5”/“LAB6” /“LAB7”*的注释相应部分。并确保编译通过。注意：为了能够正确执行*lab8*的测试应用程序，可能需对已完成的实验*1/2/3/4/5/6/7*的代码进行进一步改进。

经过meld对比发现，需要修改的部分如下：

- proc.c
- default_pmm.c
- pmm.c
- swap_fifo.c
- vmm.c
- trap.c
- sche.c
- monitor.

- check_sync.c

将上述文件复制到lab8文件夹即可。



同时经过对比发现并没有其他需要修改的地方。

练习1:完成读文件操作的实现（需要编码）

> 首先了解打开文件的处理流程，然后参考本实验后续的文件读写操作的过程分析，编写在*sfs_inode.c*中*sfs_io_nolock*读文件中数据的实现代码。
>
> 请在实验报告中给出设计实现”*UNIX*的*PIPE*机制“的概要设方案，鼓励给出详细设计方案

1、前置知识

操作系统中负责管理和存储可长期保存数据的软件功能模块称为文件系统。在本次试验中，主要侧重文件系统的设计实现和对文件系统执行流程的分析与理解。

ucore 的文件系统模型源于 Havard 的 OS161 的文件系统和 Linux 文件系统。但其实这二者都是源于传统的 UNIX 文件系统设计。

①4个文件系统抽象概念

UNIX 提出了四个文件系统抽象概念：文件(file)、目录项(dentry)、索引节点(inode)和安装点(mount point)。

- 文件：UNIX 文件中的内容可理解为是一有序字节 buffer，文件都有一个方便应用程序识别的文件名称（也称文件路径名）。典型的文件操作有读、写、创建和删除等。
- 目录项：目录项不是目录，而是目录的组成部分。在 UNIX 中目录被看作一种特定的文件，而目录项是文件路径中的一部分。如一个文件路径名是“/test/testfile”，则包含的目录项为：根目录“/”，目录“test”和文件“testfile”，这三个都是目录项。一般而言，目录项包含目录项的名字（文件名或目录名）和目录项的索引节点（见下面的描述）位置。
- 索引节点：UNIX 将文件的相关元数据信息（如访问控制权限、大小、拥有者、创建时间、数据内容等等信息）存储在一个单独的数据结构中，该结构被称为索引节点。
- 安装点：在 UNIX 中，文件系统被安装在一个特定的文件路径位置，这个位置就是安装点。所有的已安装文件系统都作为根文件系统树中的叶子出现在系统中。安装点是一个起点，从安装点开始可以访问文件系统中的所有文件。

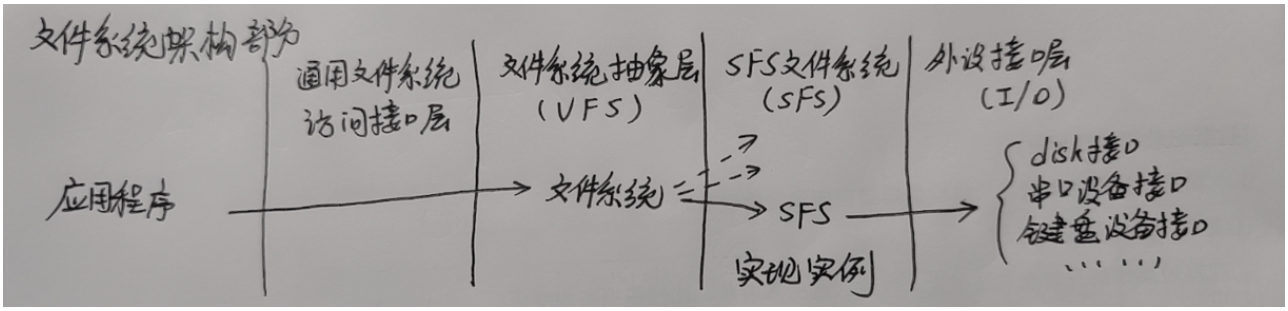
上述抽象概念形成了UNIX文件系统的逻辑数据结构，并需要通过一个具体文件系统的架构设计与实现把上述信息映射并储存到磁盘介质上，从而在具体文件系统的磁盘布局（即数据在磁盘上的物理组织）上具体体现出上述抽象概念。比如文件元数据信息存储在磁盘块中的索引节点上。当文件被载入内存时，内核需要使用磁盘块中的索引点来构造内存中的索引节点。

②4个文件系统架构组成部分

ucore模仿了UNIX的文件系统设计，ucore的文件系统架构主要由四部分组成：

- 通用文件系统访问接口层：该层提供了一个从用户空间到文件系统的标准访问接口。这一层访问接口让应用程序能够通过一个简单的接口获得ucore内核的文件系统服务。
- 文件系统抽象层：向上提供一个一致的接口给内核其他部分（文件系统相关的系统调用实现模块和其他内核功能模块）访问。向下提供一个同样的抽象函数指针列表和数据结构屏蔽不同文件系统的实现细节。
- Simple FS文件系统层：一个基于索引方式的简单文件系统实例。向上通过各种具体函数实现以对应文件系统抽象层提出的抽象函数。向下访问外设接口
- 外设接口层：向上提供device访问接口屏蔽不同硬件细节。向下实现访问各种具体设备驱动的接口，比如disk设备接口/串口设备接口/键盘设备接口等。

对照上面的层次我们再大致介绍一下文件系统的访问处理过程，加深对文件系统的总体理解。假如应用程序操作文件（打开/创建/删除/读写），首先需要通过文件系统的通用文件系统访问接口层给用户空间提供的访问接口进入文件系统内部，接着由文件系统抽象层把访问请求转发给某一具体文件系统（比如SFS文件系统），具体文件系统（Simple FS文件系统层）把应用程序的访问请求转化为对磁盘上的block的处理请求，并通过外设接口层交给磁盘驱动例程来完成具体的磁盘操作。结合用户态写文件函数write的整个执行过程，我们可以比较清楚地看出ucore文件系统架构的层次和依赖关系。



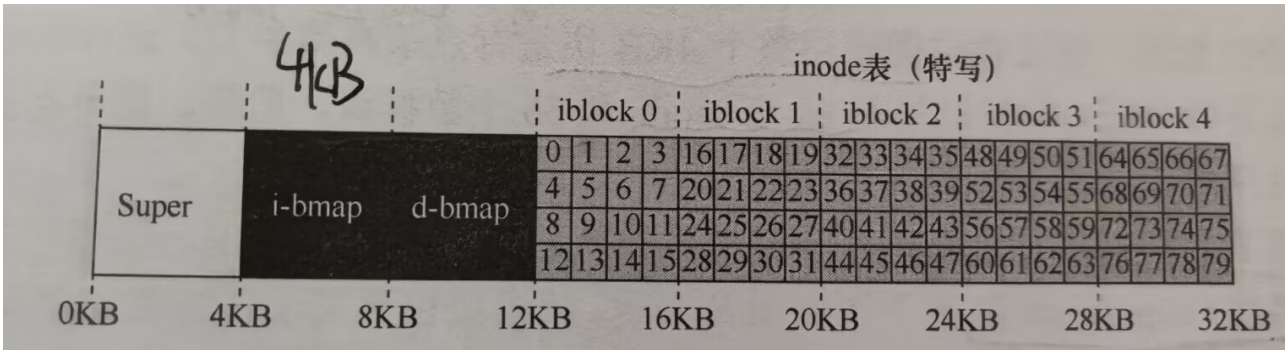
其中，文件和目录是给应用程序看到的一个抽象（用户看到的逻辑界面）。

③4类主要的数据结构

ucore 中的文件系统架构包含四类主要的数据结构：

- 超级块（SuperBlock），它主要从文件系统的全局角度描述特定文件系统的全局信息。它的作用范围是整个OS空间。
- 索引节点（inode）：它主要从文件系统的单个文件的角度它描述了文件的各种属性和数据所在位置。它的作用范围是整个OS空间。
- 目录项（dentry）：它主要从文件系统的文件路径的角度描述了文件路径中的特定目录。它的作用范围是整个 OS 空间。
- 文件（file），它主要从进程的角度描述了一个进程在访问文件时需要了解的文件标识，文件读写的位置，文件引用情况等信息。它的作用范围是某一具体进程。

文件系统，会将磁盘上的文件（程序）读取到内存里面来，在用户空间里面变成进程去进一步执行或其他操作。通过一系列系统调用完成这个过程。



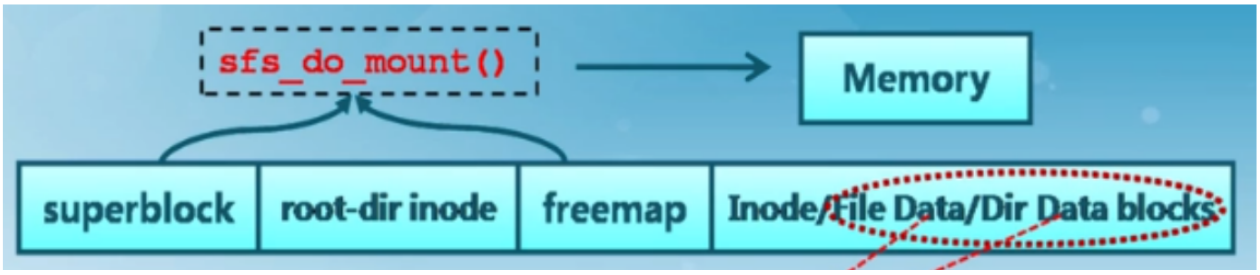
(OS书上对于Super块和inode的介绍)

2、代码级分析VFS和SFS文件系统

（1）SFS（SimpleFileSystem）

①**SFS**抽象定义

上图所示的是一个 SFS 的文件系统：



下面是定义的代码

```
//定义在kern/fs/sfs/sfs.h，83--94行
struct sfs_fs {
    struct sfs_super super;                 /* on-disk superblock */
    struct device *dev;                     /* device mounted on */
    struct bitmap *freemap;                 /* blocks in use are mared
0 */
    //前三项对应硬盘文件布局的全局信息
    bool super_dirty;                       /* true if super/freemap
modified */
    void *sfs_buffer;                       /* buffer for non-block
aligned io */
    semaphore_t fs_sem;                     /* semaphore for fs */
    semaphore_t io_sem;                     /* semaphore for io */
    semaphore_t mutex_sem;                  /* semaphore for
link/unlink and rename */
    list_entry_t inode_list;                /* inode linked-list */
    list_entry_t *hash_list;                /* inode hash linked-list
*/
};
```

下面是文件布局的数据结构：

②超级块 **super_block**

```
//定义在kern/fs/sfs/sfs.h，40--45行
struct sfs_super {
    //超级块，是一个文件系统的全局角度描述特定文件系统的全局信息。
    //这里面定义了标识符 magic、总块数 blocks、空闲块数 unused_blocks 和一些关于 SFS 的信息，通常是字符串。
    uint32_t magic;                         /* magic number, should be SFS_MAGIC */
    uint32_t blocks;                        /* # of blocks in fs */
    uint32_t unused_blocks;                 /* # of unused blocks in fs */
    char info[SFS_MAX_INFO_LEN + 1];        /* infomation for sfs  */
};
```

③根目录结构 **root_dir**

```
//定义在kern/fs/sfs/sfs.h，48--57行
struct sfs_disk_inode {
    //iNode 是从文件系统的单个文件的角度它描述了文件的各种属性和数据所在位置，相当于一个索引，
    //root_dir 是一个根目录索引，根目录表示，我们一开始访问这个文件系统可以看到的目录信息。
    //其中direct 和 indirect，分别代表根目录下的直接索引和间接索引。
    uint32_t size;                          /* size of the file (in bytes) */
    uint16_t type;                          /* one of SYS_TYPE_* above */
    uint16_t nlinks;                        /* # of hard links to this file */
    uint32_t blocks;                        /* # of blocks */
    uint32_t direct[SFS_NDIRECT];           /* direct blocks */
    uint32_t indirect;                      /* indirect blocks */
};
```

④目录项 **entry**

```
//定义在kern/fs/sfs/sfs.h，60--63行
struct sfs_disk_entry {
    uint32_t ino;                                   /* inode number */
    char name[SFS_MAX_FNAME_LEN + 1];               /* file name */
};
```

⑤**sfs_inode**

仅有硬盘文件布局还不够，SFS 毕竟是一个在硬盘之上的抽象，它还需要传递上一层过来的索引值 INODE。这里的 INODE 是 SFS 层面上的名字，而iNode 是硬盘上实际的索引。

```
//定义在kern/fs/sfs/sfs.h，69--77行
struct sfs_inode {
    struct sfs_disk_inode *din;             /* on-disk inode */
    uint32_t ino;                           /* inode number */
    bool dirty;                             /* true if inode
modified */
    int reclaim_count;                      /* kill inode if it
hits zero */
    semaphore_t sem;                        /* semaphore for din */
    list_entry_t inode_link;                /* entry for linked-
list in sfs_fs */
    list_entry_t hash_link;                 /* entry for hash
linked-list in sfs_fs */
};
```

sfs_disk_inode 是 SFS 层面上的 iNode 的一个成员，代表了这两个结构之间的上下级关系。

（2）VFS（VirtualFileSystem）

下面分析更高层的数据结构 VFS（虚拟文件系统）。

在 VFS 层中，我们需要对于虚拟的 iNode，和下一层 SFS 的 iNode 进行对接。

文件系统抽象层是把不同文件系统的对外共性接口提取出来，形成一个函数指针数组，这样，通用文件系统访问接口层只需访问文件系统抽象层，而不需关心具体文件系统的实现细节和接口。

①**VFS**的抽象定义

```
//定义在kern/fs/vfs/vfs.h，35--46行
struct fs {
    union {
        struct sfs_fs __sfs_info;
    } fs_info;                                      // filesystem-
specific data
    enum {
        fs_type_sfs_info,
    } fs_type;                                      // filesystem
type
    int (*fs_sync)(struct fs *fs);                  // Flush all
dirty buffers to disk
    struct inode *(*fs_get_root)(struct fs *fs);    // Return root
inode of filesystem.
    int (*fs_unmount)(struct fs *fs);               // Attempt
unmount of filesystem.
    void (*fs_cleanup)(struct fs *fs);              // Cleanup of
filesystem.???
};
```

主要是一些函数指针用于处理 VFS 的操作。

②文件结构

```
//定义在kern/fs/file.c，14--24行
struct file {
    enum {
        FD_NONE, FD_INIT, FD_OPENED, FD_CLOSED,
    } status;                       //访问文件的执行状态
    bool readable;                  //文件是否可读
    bool writable;                  //文件是否可写
    int fd;                         //文件在 filemap 中的索引值
    off_t pos;                      //访问文件的当前位置
    struct inode *node;             //该文件对应的内存 inode 指针
    atomic_t open_count;            //打开此文件的次数
};
```

在 file 基础之上还有一个管理所有 file 的数据结构 file_struct，如下：

```
//定义在kern/fs/fs.h，25--30行
struct files_struct {
    struct inode *pwd;          //当前工作目录
    struct file *fd_array;      //已经打开的文件对应的数组
    int files_count;            //打开的文件个数
};
```

③**VFS** 的索引 **inode**

```
//定义在kern/fs/vfs/inode.h，29--42行
/*
inode 数据结构是位于内存的索引节点，把不同文件系统的特定索引节点信息(甚至不能算是一个索引节点)统一封装起来，避免了进程直接访问具体文件系统
*/
struct inode {
    union {                                 //包含不同文件系统特定 inode 信息的 union 域
        struct device __device_info;        //设备文件系统内存 inode 信息
        struct sfs_inode __sfs_inode_info;  //SFS 文件系统内存 inode 信息
    } in_info;
    enum {
        inode_type_device_info = 0x1234,
        inode_type_sfs_inode_info,
    } in_type;                              //此 inode 所属文件系统类型
    atomic_t ref_count;                     //此 inode 的引用计数
    atomic_t open_count;                    //打开此 inode 对应文件的个数
    struct fs *in_fs;                       //抽象的文件系统,包含访问文件系统的函数指针
    const struct inode_ops *in_ops;         //抽象的 inode 操作,包含访问 inode 的函数指针
};
```

我们看到在 VFS 层面的 iNode 值，包含了 SFS 和硬件设备 device 的情况。

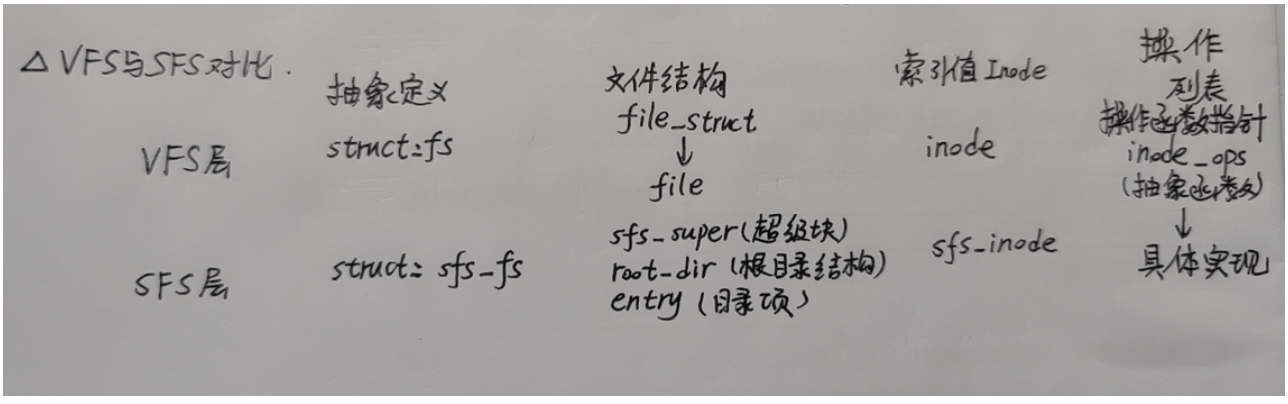
④**inode** 的操作函数指针列表

```
//定义在kern/fs/vfs/inode.h，169--186行
//实现对常规文件、目录、设备文件所有操作的一个抽象函数
struct inode_ops {
    unsigned long vop_magic;
    int (*vop_open)(struct inode *node, uint32_t open_flags);
    int (*vop_close)(struct inode *node);
    int (*vop_read)(struct inode *node, struct iobuf *iob);
    int (*vop_write)(struct inode *node, struct iobuf *iob);
    int (*vop_fstat)(struct inode *node, struct stat *stat);
    int (*vop_fsync)(struct inode *node);
    int (*vop_namefile)(struct inode *node, struct iobuf *iob);
    int (*vop_getdirentry)(struct inode *node, struct iobuf *iob);
    int (*vop_reclaim)(struct inode *node);
    int (*vop_gettype)(struct inode *node, uint32_t *type_store);
    int (*vop_tryseek)(struct inode *node, off_t pos);
    int (*vop_truncate)(struct inode *node, off_t len);
    int (*vop_create)(struct inode *node, const char *name, bool
excl, struct inode **node_store);
    int (*vop_lookup)(struct inode *node, char *path, struct inode
**node_store);
    int (*vop_ioctl)(struct inode *node, int op, void *data);
};
```

对于某一具体的文件系统中的文件或目录，只需实现相关的函数，就可以被用户进程访问具体的文件了，且用户进程无需了解具体文件系统的实现细节。

★（3）SFS与VFS对比

根据上述（1）（2）可总结如下



3、打开文件的处理流程

总体分析

接下来分析打开一个文件的详细处理的流程。

例如某一个应用程序需要操作文件（增删读写等），首先需要通过文件系统的通用文件系统访问接口层给用户空间提供的访问接口进入文件系统内部，接着由文件系统抽象层把访问请求转发给某一具体文件系统(比如 Simple FS 文件系统)，然后再由具体文件系统把应用程序的访问请求转化为对磁盘上的 block 的处理请求，并通过外设接口层交给磁盘驱动例程来完成具体的磁盘操作。



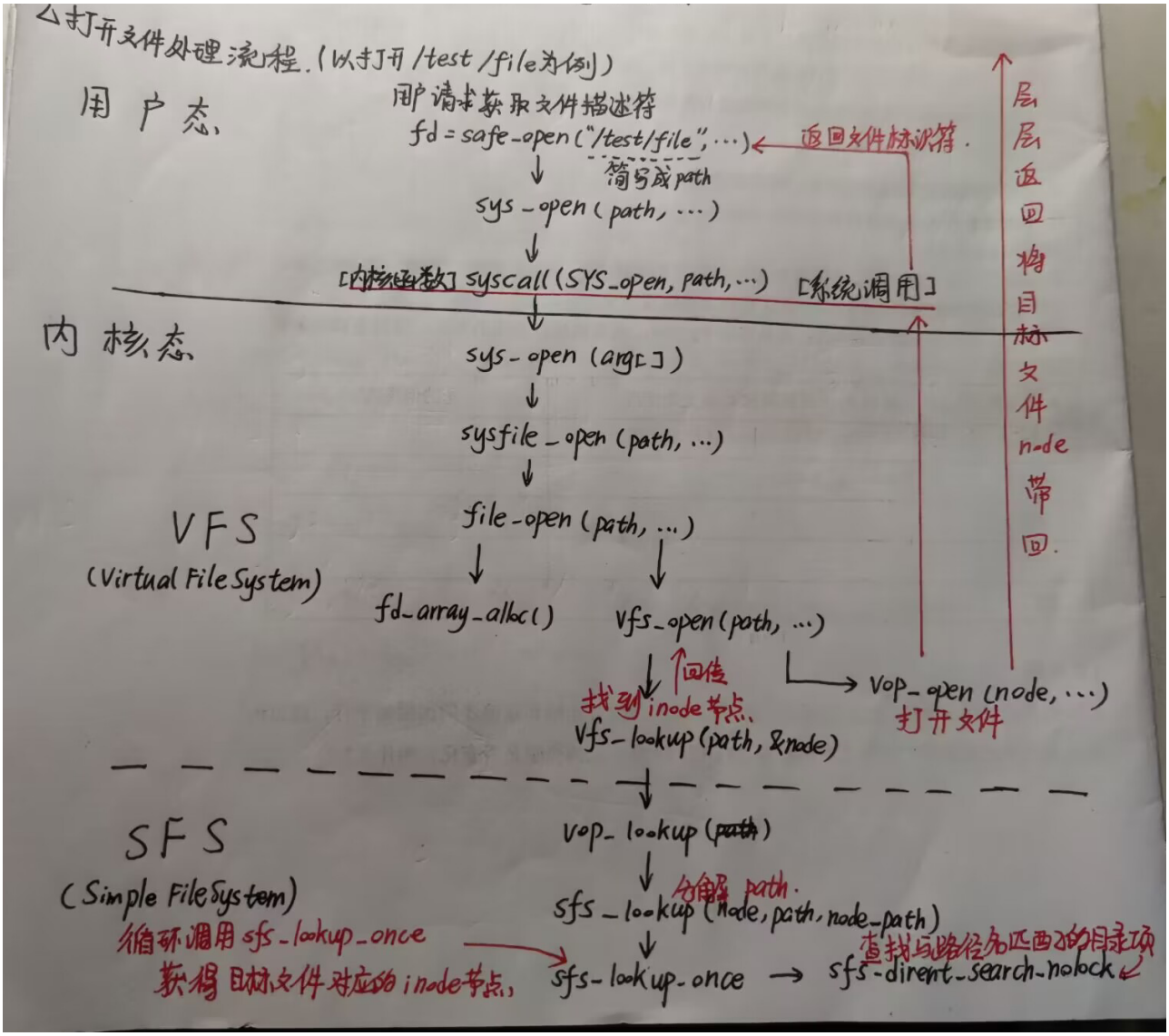
对应到我们的ucore上，具体的过程如下：

- 1、 以打开文件为例，首先用户会在进程中调用 safe_open() 函数，然后依次调用如下函数 open->sys_open->syscall，从而引发系统调用然后进入内核态，然后会由 sys_open 内核函数处理系统调用，进一步调用到内核函数 sysfile_open，然后将字符串 "/test/testfile" 拷贝到内核空间中的字符串 path 中，并进入到文件系统抽象层的处理流程完成进一步的打开文件操作中。
- 2、 在文件系统抽象层，系统会分配一个 file 数据结构的变量，这个变量其实是 current->fs_struct->filemap[] 中的一个空元素，即还没有被用来打开过文件，但

是分配完了之后还不能找到对应对应的文件结点。所以系统在该层调用了vfs_open 函数通过调用 vfs_lookup 找到 path 对应文件的 inode，然后调用vop_open函数打开文件。然后层层返回，通过执行语句 file->node=node;，就把当前进程的 current->fs_struct->filemap[fd]（即 file 所指变量）的成员变量 node 指针指向了代表文件的索引节点 node。这时返回 fd。最后完成打开文件的操作。

- 3、 在第2步中，调用了 SFS 文件系统层的 vfs_lookup 函数去寻找 node，这里在 sfs_inode.c 中我们能够知道 .vop_lookup = sfs_lookup。
- 4、看到 sfs_lookup 函数传入的三个参数，其中 node 是根目录“/”所对应的 inode 节点；path 是文件的绝对路径（例如“/test/file”），而 node_store 是经过查找获得的 file 所对应的 inode 节点。 函数以“/”为分割符，从左至右逐一分解path获得各个子目录和最终文件对应的 inode 节点。在本例中是分解出 “test” 子目录，并调用 sfs_lookup_once 函数获得 “test” 子目录对应的 inode 节点 subnode，然后循环进一步调用 sfs_lookup_once 查找以 “test” 子目录下的文件 “testfile1” 所对应的 inode 节点。当无法分解 path 后，就意味着找到了testfile1对应的 inode 节点，就可顺利返回了。
- 5、而我们再进一步观察 sfs_lookup_once 函数，它调用 sfs_dirent_search_nolock 函数来查找与路径名匹配的目录项，如果找到目录项，则根据目录项中记录的 inode 所处的数据块索引值找到路径名对应的 SFS 磁盘 inode，并读入 SFS 磁盘 inode 对的内容，创建 SFS 内存 inode。

★流程总结图



sfs_lookup

函数以 “/” 为分割符，从左至右逐一分解path获得各个子目录和最终文件对应的 inode 节点。在本例中是分解出 “test” 子目录，并调用sfs_lookup_once函数获得“test”子目录对应的 inode 节点 subnode，然后循环进一步调用 sfs_lookup_once 查找以 “test” 子目录下的文件 “testfile1” 所对应的 inode 节点。当无法分解 path 后，就意味着找到了 testfile1 对应的 inode 节点，就可顺利返回了。

```
//在kern/fs/sfs/sfs_inode.c，975--993行
static int sfs_lookup(struct inode *node, char *path, struct inode
**node_store) {
    //三个参数分别是根目录/的inode节点，文件的绝对路径，目标inode节点
    //具体地说，node 是根目录 “/” 所对应的 inode 节点；path 是文件的绝对路径
（例如 “/test/file”），而 node_store 是经过查找获得的file所对应的inode节点
struct sfs_fs *sfs = fsop_info(vop_fs(node), sfs);
assert(*path != '\0' && *path != '/');
//以“/”为分割符，从左至右分解path获得各子目录和最终文件对应的inode节点。
```

```
vop_ref_inc(node);
struct sfs_inode *sin = vop_info(node, sfs_inode);
if (sin->din->type != SFS_TYPE_DIR) {
        vop_ref_dec(node);
        return -E_NOTDIR;
}
struct inode *subnode;
int ret = sfs_lookup_once(sfs, sin, path, &subnode, NULL);
//循环进一步调用sfs_lookup_once查找以“test”子目录下的文件“testfile1”所对应的inode节点。

vop_ref_dec(node);
if (ret != 0) {
        return ret;
}
*node_store = subnode;
//当无法分解path后，就意味着找到了需要对应的inode节点，就可顺利返回了。
return 0;
}
```

sfs_lookup_once

再进一步观察 sfs_lookup_once 函数，它调用 sfs_dirent_search_nolock 函数来查找与路径名匹配的目录项，如果找到目录项，则根据目录项中记录的 inode 所处的数据块索引值找到路径名对应的 SFS 磁盘 inode，并读入 SFS 磁盘 inode 对的内容，创建 SFS 内存 inode。

```
//在kern/fs/sfs/sfs_inode.c，498--512行
static int sfs_lookup_once(struct sfs_fs *sfs, struct sfs_inode
*sin, const char *name, struct inode **node_store, int *slot) {
    int ret;
    uint32_t ino;
    lock_sin(sin);
    {   // find the NO. of disk block and logical index of file
entry
        ret = sfs_dirent_search_nolock(sfs, sin, name, &ino, slot,
NULL);
    }
    unlock_sin(sin);
    if (ret == 0) {
        // load the content of inode with the the NO. of disk block
        ret = sfs_load_inode(sfs, node_store, ino);
```

```
    }
    return ret;
}
```

★4、读文件

ucore 文件系统中，是这样处理读写硬盘操作的：

- （1）首先是应用程序发出请求，请求硬盘中写数据或读数据，应用程序通过 FS syscall 接口执行系统调用，获得 ucore 操作系统关于文件的一些服务；
- （2）之后，一旦操作系统内系统调用得到了请求，就会到达 VFS 层面（虚拟文件系统），包含很多部分比如文件接口、目录接口等，是一个抽象层面，它屏蔽底层具体的文件系统；
- （3）VFS 如果得到了处理，那么 VFS 会将这个 iNode 传递给 SimpleFS，注意，此时，VFS 中的 iNode 还是一个抽象的结构，在 SimpleFS 中会转化为一个具体的 iNode；
- （4）通过该 iNode 经过 IO 接口对于磁盘进行读写。

（与打开的方式经过的层次基本类似）

sfs_io_nolock函数（执行读写）

sfs_io_nolock函数主要用来将磁盘中的一段数据读入到内存中或者将内存中的一段数据写入磁盘。

该函数会进行一系列的边缘检查，检查访问是否越界、是否合法。之后将具体的读/写操作使用函数指针统一起来，统一成针对整块的操作。然后完成不落在整块数据块上的读/写操作，以及落在整块数据块上的读写。下面是该函数的代码(包括补充的代码，其中将有较为详细的注释)

```
//在kern/fs/sfs/sfs_inode.c，552--644行
static int
sfs_io_nolock(struct sfs_fs *sfs, struct sfs_inode *sin, void *buf,
off_t offset, size_t *alenp, bool write)
{   //创建一个磁盘索引节点指向要访问文件的内存索引节点
    struct sfs_disk_inode *din = sin->din;
    assert(din->type != SFS_TYPE_DIR);
    //确定读取的结束位置
    off_t endpos = offset + *alenp, blkoff;
    *alenp = 0;
    //检测一系列的边缘，避免非法访问
```

```
    if (offset < 0 || offset >= SFS_MAX_FILE_SIZE || offset >
endpos) {
        return -E_INVAL;
    }
    if (offset == endpos) {
        return 0;
    }
    if (endpos > SFS_MAX_FILE_SIZE) {
        endpos = SFS_MAX_FILE_SIZE;
    }
    if (!write) {
        if (offset >= din->size) {
            return 0;
        }
        if (endpos > din->size) {
            endpos = din->size;
        }
    }
    int (*sfs_buf_op)(struct sfs_fs *sfs, void *buf, size_t len,
uint32_t blkno, off_t offset);
    int (*sfs_block_op)(struct sfs_fs *sfs, void *buf, uint32_t
blkno, uint32_t nblks);
    //确定是读操作还是写操作，并确定相应的系统函数
    if (write) {
        sfs_buf_op = sfs_wbuf, sfs_block_op = sfs_wblock;//执行写操作
    }
    else {
        sfs_buf_op = sfs_rbuf, sfs_block_op = sfs_rblock;//执行读操作
    }
    //初始返回值设定与初始化
    int ret = 0;
    size_t size, alen = 0;
    uint32_t ino;
    uint32_t blkno = offset / SFS_BLKSIZE;          // The NO. of
Rd/Wr begin block
    uint32_t nblks = endpos / SFS_BLKSIZE - blkno;  // The size of
Rd/Wr blocks
//--------------------------补充部分---------------------------------
-------------
// 判断被需要操作的区域的数据块中的第一块是否是完全被覆盖的，
// 如果不是，则需要调用非整块数据块进行读或写的函数来完成相应操作
      if ((blkoff = offset % SFS_BLKSIZE) != 0)  {
```

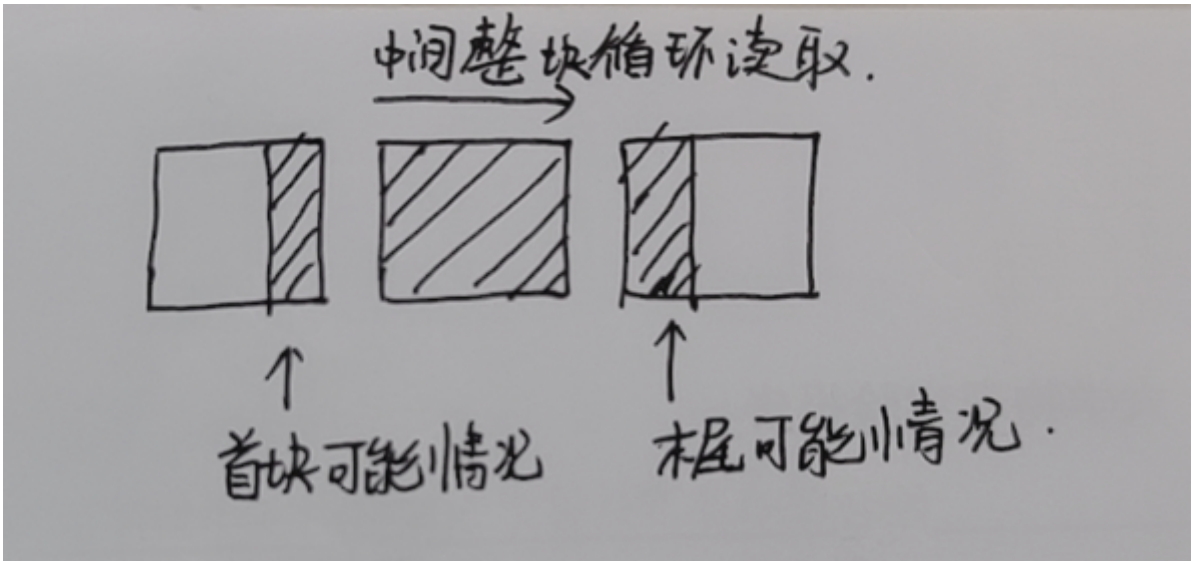
```
        // 第一块数据块中进行操作的偏移量
          blkoff = offset % SFS_BLKSIZE;
        // 第一块数据块中进行操作的数据长度
        size = (nblks != 0) ? (SFS_BLKSIZE - blkoff) : (endpos -
offset);
       //获取这些数据块对应到磁盘上的数据块的编号
        if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) !=
0) {
            goto out;
        }
        //对数据块进行读或写操作
        if ((ret = sfs_buf_op(sfs, buf, size, ino, blkoff)) != 0) {
            goto out;
        }
        //已经完成读写的数据长度
        alen += size;//读取到的长度
        if (nblks == 0) {//大小小于一个块，读取结束
            goto out;
        }
        buf += size, blkno++; nblks--;
    }

    //读取中间部分的数据，将其分为大小为size的块，然后一块一块操作，直至完成
    size = SFS_BLKSIZE;
    while (nblks != 0) //循环读取整块
    {
        if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) !=
0) {
            goto out;
        }
        //对数据块进行读或写操作
        if ((ret = sfs_block_op(sfs, buf, ino, 1)) != 0) {
            goto out;
        }
        //更新相应的变量
        alen += size, buf += size, blkno++, nblks--;
    }

    // 最后一页，可能出现不对齐的现象:
    if ((size = endpos % SFS_BLKSIZE) != 0)
    {
         // 获取该数据块对应到磁盘上的数据块的编号
```

```
        if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) !=
0) {
            goto out;
        }
        // 进行非整块的读或者写操作
        if ((ret = sfs_buf_op(sfs, buf, size, ino, 0)) != 0) {//偏移
为0，大小为size
            goto out;
        }
        alen += size;
    }
out:
    *alenp = alen;
    if (offset + alen > sin->din->size) {
        //如果大于磁盘数据块已有大小
        sin->din->size = offset + alen;
        //更新大小
        sin->dirty = 1;
        //标记脏位
    }
    return ret;
}
```

总的来说就是分为三部分来读取文件。



每次通过`sfs_bmap_load_nolock`函数获取文件索引编号，然后调用`sfs_buf_op`或者`sfs_block_op`完成实际的文件读写操作。

★5、设计实现”UNIX的PIPE机制“的概要设方案

★1、管道（**PIPE**）机制

管道是一种最基本的IPC机制，作用于有血缘关系的进程之间，完成数据传递。调用pipe系统函数即可创建一个管道。有如下特质：

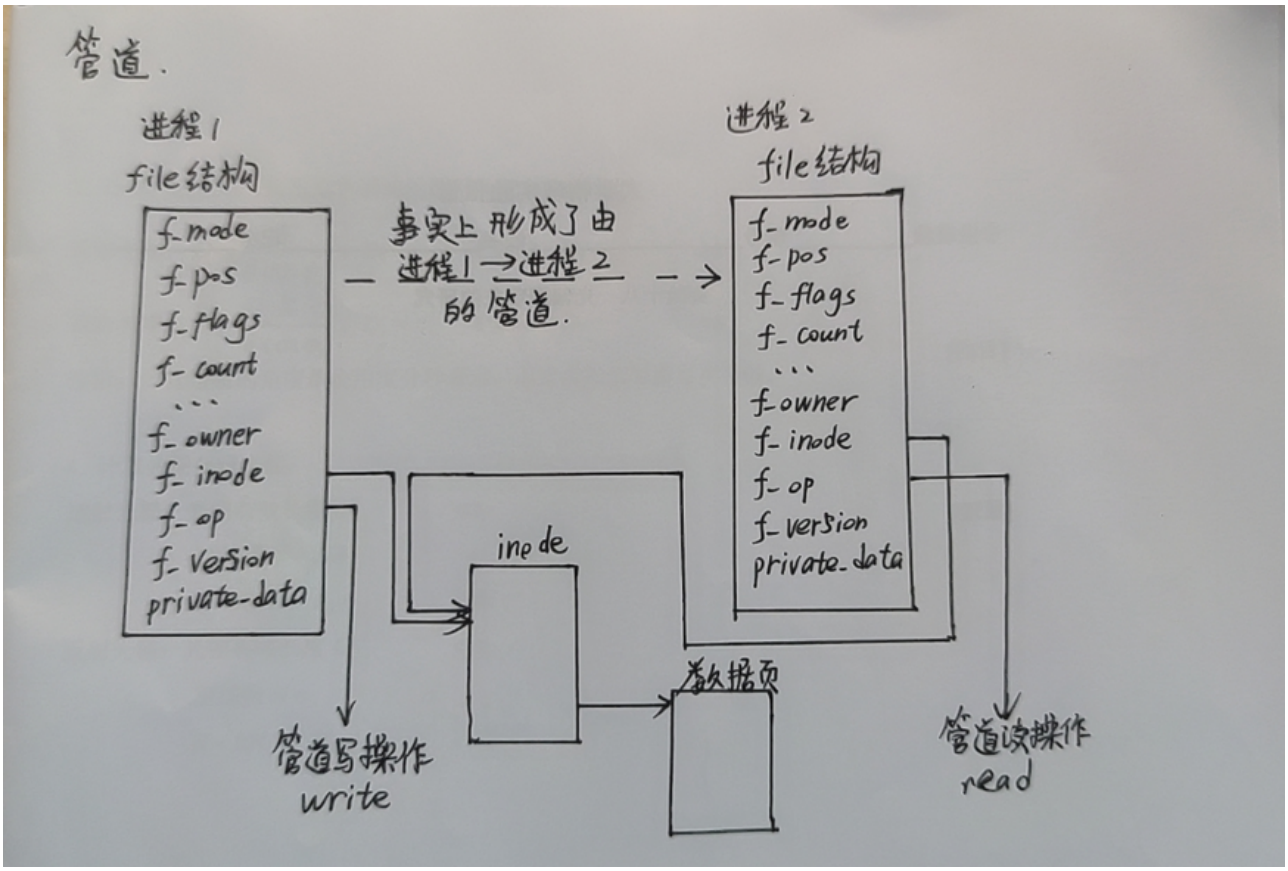
- 其本质是一个伪文件(实为内核缓冲区)
- 由两个文件描述符引用，一个表示读端，一个表示写端。
- 规定数据从管道的写端流入管道，从读端流出。

管道的原理:

- 管道实为内核使用环形队列机制，借助内核缓冲区(4k)实现。

管道的局限性：

- 数据自己读不能自己写。
- 数据一旦被读走，便不在管道中存在，不可反复读取。
- 由于管道采用半双工通信方式。因此，数据只能在一个方向上流动。
- 只能在有公共祖先的进程间使用管道。



2、设计方案

为了实现 UNIX 的 PIPE 机制，可以考虑在磁盘上保留一部分空间或者是一个特定的文件来作为 pipe 机制的缓冲区，接下来将说明如何完成对 pipe 机制的支持：

- 首先我们需要在磁盘上保留一定的区域用来作为PIPE机制的缓冲区，或者创建一个文件为PIPE机制服务
- 当某两个进程之间要求建立管道，假定将进程 A 的标准输出作为进程B的标准输入，那么可以在这两个进程的进程控制块上新增变量来记录进程的这种属性；并且同时生成一个临时的文件，并将其在进程A, B中打开；
- 当进程 A 使用标准输出进行 write 系统调用的时候，通过PCB中的变量可以知道，需要将这些标准输出的数据输出到先前准备的临时文件缓冲区中去；
- 当进程 B 使用标准输入的时候进行 read 系统调用的时候，根据其PCB中的信息可以知道，需要从上述的临时文件中读取数据；
- 至此完成了对 pipe 机制的设计；

事实上，由于在真实的文件系统和用户之间还由一层虚拟文件系统，因此我们也可以不把数据缓冲在磁盘上，而是直接保存在内存中，然后完成一个根据虚拟文件系统的规范完成一个虚拟的 pipe 文件，然后进行输入输出的时候只要对这个文件进行操作即可。

练习2: 完成基于文件系统的执行程序机制的实现（需要编码）

> 改写*proc.c*中的*load_icode*函数和其他相关函数，实现基于文件系统的执行程序机制。执行：*make qemu*。如果能看看到*sh*用户程序的执行界面，则基本成功了。如果在*sh*用户界面上可以执行*"ls","hello"*等其他放置在*sfs*文件系统中的其他执行程序，则可以认为本实验基本成功。
>
> 请在实验报告中给出设计实现基于*"UNIX*的硬链接和软链接机制"的概要设方案，鼓励给出详细设计方案

可以在 Lab 7 的基础上进行修改，读 elf 文件变成从磁盘上读，而不是直接在内存中读。

需要改写的主要有三处：alloc_proc函数，load_icode函数，do_fork函数。

1、 alloc_proc 函数

由于在lab8引入了文件系统，每个进程的进程控制块都添加了一个指针指向一个fs_struct，记录进程相关的文件信息。因此在分配进程控制块时要对该指针进行初始化。

修改之后 alloc_proc 函数如下：（增加一行，kern/process/proc.c，136行）

```
proc->filesp = NULL;      //初始化fs中的进程控制结构
```

所以完整的 alloc_proc 函数的实现如下：

```
//LAB8:EXERCISE2 YOUR CODE HINT:need add some code to init fs in
proc_struct, ...
static struct proc_struct *alloc_proc(void) {
    struct proc_struct *proc = kmalloc(sizeof(struct proc_struct));
    if (proc != NULL) {
        proc->state = PROC_UNINIT; //进程状态为为初始化
        proc->pid = -1;            //进程ID为-1
        proc->runs = 0;            //进程运行时间为0
        proc->kstack = 0;          //内核栈为0
        proc->need_resched = 0;    //进程不需要调度
        proc->parent = NULL; //父进程为空
        proc->mm = NULL; //内存管理为空
        memset(&(proc->context), 0, sizeof(struct context));
        proc->tf = NULL; //中断帧为空
        proc->cr3 = boot_cr3; //cr3寄存器
        proc->flags = 0; //标记
        memset(proc->name, 0, PROC_NAME_LEN);
        proc->wait_state = 0; //等待状态
        proc->cptr = proc->optr = proc->yptr = NULL; //相关指针初始化
        proc->rq = NULL; //运行队列
        list_init(&(proc->run_link)); //运行队列链表
        proc->time_slice = 0; //进程运行的时间片
        proc->lab6_run_pool.left = proc->lab6_run_pool.right =
proc->lab6_run_pool.parent = NULL; //进程池
        proc->lab6_stride = 0;
        proc->lab6_priority = 0; //优先级
        proc->filesp = NULL;  //初始化fs中的进程控制结构
    }
    return proc;
}
```

2、do_fork函数

相应的，do_fork函数也要修改，子进程需要复制父进程的文件相关信息。为此需要新创建一个fs_struct结构，并将父进程fs_struct中的内容复制到子进程的fs_struct中。在proc.c中已经提供了实现进程文件信息复制的函数copy_files，在do_fork中调用该函数让子进程复制父进程的fs_struct，如果没有复制成功，要处理已分配的fs_struct。

```
//复制fs_struct
static int
```

```
copy_files(uint32_t clone_flags, struct proc_struct *proc);
//减少fs_struct的引用计数，计数为0则删除
static void
put_files(struct proc_struct *proc);
//do_fork中的修改
int
do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe
*tf) {
    //分配proc_struct，分配内核栈......
    //将当前进程的fs复制给fork子进程
    if (copy_files(clone_flags, proc) != 0) {
        goto bad_fork_cleanup_kstack;
    }
    //复制内存空间等......
fork_out:
    return ret;
bad_fork_cleanup_fs:  //复制失败处理
    put_files(proc);
bad_fork_cleanup_kstack:
    put_kstack(proc);
bad_fork_cleanup_proc:
    kfree(proc);
    goto fork_out;
}
```

3、load_icode函数

然后就是要实现 `load_icode` 函数，具体的实现及注释如下所示：

```
static int load_icode(int fd, int argc, char **kargv) {

    /* (1) create a new mm for current process
     * (2) create a new PDT, and mm->pgdir= kernel virtual addr of
PDT
     * (3) copy TEXT/DATA/BSS parts in binary to memory space of
process
     *    (3.1) read raw data content in file and resolve elfhdr
     *    (3.2) read raw data content in file and resolve proghdr
based on info in elfhdr
     *    (3.3) call mm_map to build vma related to TEXT/DATA
```

```
     *    (3.4) callpgdir_alloc_page to allocate page for
TEXT/DATA, read contents in file
     *          and copy them into the new allocated pages
     *    (3.5) callpgdir_alloc_page to allocate pages for BSS,
memset zero in these pages
     * (4) call mm_map to setup user stack, and put parameters into
user stack
     * (5) setup current process's mm, cr3, reset pgidr (using lcr3
MARCO)
     * (6) setup uargc and uargv in user stacks
     * (7) setup trapframe for user environment
     * (8) if up steps failed, you should cleanup the env.
     */
    assert(argc >= 0 && argc <= EXEC_MAX_ARG_NUM);
    //(1)建立内存管理器
    // 判断当前进程的 mm 是否已经被释放掉了
    if (current->mm != NULL) {    //要求当前内存管理器为空
        panic("load_icode: current->mm must be empty.\n");
    }

    int ret = -E_NO_MEM;     // E_NO_MEM 代表因为存储设备产生的请求错误
    struct mm_struct *mm;  //建立内存管理器
    if ((mm = mm_create()) == NULL) {// 为进程创建一个新的 mm
        goto bad_mm;
    }


    //(2)建立页目录
    if (setup_pgdir(mm) != 0) {// 进行页表项的设置
        goto bad_pgdir_cleanup_mm;//创建mm失败，进入错误处理
    }
    struct Page *page;//建立页表


    //(3)从文件加载程序到内存
    struct elfhdr __elf, *elf = &__elf;
    // 从磁盘上读取出 ELF 可执行文件的 elf-header
    if ((ret = load_icode_read(fd, elf, sizeof(struct elfhdr), 0))
!= 0) {//读取 elf 文件头
        goto bad_elf_cleanup_pgdir;
    }
```

```
    if (elf->e_magic != ELF_MAGIC) {// 判断该 ELF 文件是否合法（依据
MAGIC魔数）
        ret = -E_INVAL_ELF;
        goto bad_elf_cleanup_pgdir;
    }

    struct proghdr __ph, *ph = &__ph;
    uint32_t vm_flags, perm, phnum;
    // 根据 elf-header 中的信息，找到每一个 program header
    for (phnum = 0; phnum < elf->e_phnum; phnum ++) {
        //e_phnum 代表程序段入口地址数目，即多少各段
        off_t phoff = elf->e_phoff + sizeof(struct proghdr) *
phnum;
        //循环读取程序的每个段的头部
        if ((ret = load_icode_read(fd, ph, sizeof(struct proghdr),
phoff)) != 0) {// 读取program header
            goto bad_cleanup_mmap;
        }
        if (ph->p_type != ELF_PT_LOAD) {
            continue ;
        }
        if (ph->p_filesz > ph->p_memsz) {
            ret = -E_INVAL_ELF;
            goto bad_cleanup_mmap;
        }
        if (ph->p_filesz == 0) {
            continue ;
        }
        vm_flags = 0, perm = PTE_U;//建立虚拟地址与物理地址之间的映射
        if (ph->p_flags & ELF_PF_X) vm_flags |= VM_EXEC;// 根据 ELF
文件中的信息，对各个段的权限进行设置
        if (ph->p_flags & ELF_PF_W) vm_flags |= VM_WRITE;
        if (ph->p_flags & ELF_PF_R) vm_flags |= VM_READ;
        if (vm_flags & VM_WRITE) perm |= PTE_W;
        if ((ret = mm_map(mm, ph->p_va, ph->p_memsz, vm_flags,
NULL)) != 0) {// 将这些段的虚拟内存地址设置为合法的
            goto bad_cleanup_mmap;
        }
        off_t offset = ph->p_offset;
        size_t off, size;
        uintptr_t start = ph->p_va, end, la = ROUNDDOWN(start,
PGSIZE);
```

```
        ret = -E_NO_MEM;

        //复制数据段和代码段
        end = ph->p_va + ph->p_filesz;      //计算数据段和代码段终止地址
        while (start < end) {
            if ((page = pgdir_alloc_page(mm->pgdir, la, perm)) ==
NULL) {// 为 TEXT/DATA 段逐页分配物理内存空间
                ret = -E_NO_MEM;
                goto bad_cleanup_mmap;
            }
            off = start - la, size = PGSIZE - off, la += PGSIZE;
            if (end < la) {
                size -= la - end;
            }
            //每次读取size大小的块，直至全部读完
            if ((ret = load_icode_read(fd, page2kva(page) + off,
size, offset)) != 0) {       //load_icode_read 通过 sysfile_read 函数
实现文件读取，将磁盘上的 TEXT/DATA 段读入到分配好的内存空间中去
                goto bad_cleanup_mmap;
            }
            start += size, offset += size;
        }
        //建立BSS段
        end = ph->p_va + ph->p_memsz;    //同样计算终止地址

        if (start < la) {// 如果存在 BSS 段，并且先前的 TEXT/DATA 段分配
的最后一页没有被完全占用，则剩余的部分被BSS段占用，因此进行清零初始化
            if (start == end) {
                continue ;
            }
            off = start + PGSIZE - la, size = PGSIZE - off;
            if (end < la) {
                size -= la - end;
            }
            memset(page2kva(page) + off, 0, size);
            start += size;
            assert((end < la && start == end) || (end >= la &&
start == la));
        }
```

```
        while (start < end) {// 如果 BSS 段还需要更多的内存空间的话，进一步进行分配
            if ((page = pgdir_alloc_page(mm->pgdir, la, perm)) == NULL) {// 为 BSS 段分配新的物理内存页
                ret = -E_NO_MEM;
                goto bad_cleanup_mmap;
            }
            off = start - la, size = PGSIZE - off, la += PGSIZE;
            if (end < la) {
                size -= la - end;
            }
            //每次操作 size 大小的块
            memset(page2kva(page) + off, 0, size);// 将分配到的空间清零初始化
            start += size;
        }
    }
    // 关闭传入的文件，因为在之后的操作中已经不需要读文件了
    sysfile_close(fd);//关闭文件，加载程序结束


    //(4)建立相应的虚拟内存映射表
    vm_flags = VM_READ | VM_WRITE | VM_STACK;// 设置用户栈的权限
    if ((ret = mm_map(mm, USTACKTOP - USTACKSIZE, USTACKSIZE, vm_flags, NULL)) != 0) {
        // 将用户栈所在的虚拟内存区域设置为合法的
        goto bad_cleanup_mmap;
    }
    assert(pgdir_alloc_page(mm->pgdir, USTACKTOP-PGSIZE , PTE_USER) != NULL);
    assert(pgdir_alloc_page(mm->pgdir, USTACKTOP-2*PGSIZE , PTE_USER) != NULL);
    assert(pgdir_alloc_page(mm->pgdir, USTACKTOP-3*PGSIZE , PTE_USER) != NULL);
    assert(pgdir_alloc_page(mm->pgdir, USTACKTOP-4*PGSIZE , PTE_USER) != NULL);


    //(5)设置用户栈
    mm_count_inc(mm);
    // 切换到用户的内存空间，这样的话后文中在栈上设置参数部分的操作将大大简化，因为具体因为空间不足而导致的分配物理页的操作已经交由page fault处理了，是完全透明的
```

```
    current->mm = mm;
    current->cr3 = PADDR(mm->pgdir);
    lcr3(PADDR(mm->pgdir));

    //(6)处理用户栈中传入的参数，其中 argc 对应参数个数，uargv[] 对应参数的具体内容的地址
    uint32_t argv_size=0, i;
    for (i = 0; i < argc; i ++) {// 先算出所有参数加起来的长度
        argv_size += strnlen(kargv[i],EXEC_MAX_ARG_LEN + 1)+1;
    }

    uintptr_t stacktop = USTACKTOP - (argv_size/sizeof(long)+1)*sizeof(long);
    char** uargv=(char **)(stacktop  - argc * sizeof(char *));

    argv_size = 0;
    for (i = 0; i < argc; i ++) {          //将所有参数取出来放置 uargv
        uargv[i] = strcpy((char *)(stacktop + argv_size ), kargv[i]);
        argv_size +=  strnlen(kargv[i],EXEC_MAX_ARG_LEN + 1)+1;
    }

    stacktop = (uintptr_t)uargv - sizeof(int);    //计算当前用户栈顶
    *(int *)stacktop = argc;


    //(7)设置进程的中断帧
    struct trapframe *tf = current->tf;// 设置中断帧
    memset(tf, 0, sizeof(struct trapframe));//初始化 tf，设置中断帧
    tf->tf_cs = USER_CS;// 需要返回到用户态，因此使用用户态的数据段和代码段的选择子
    tf->tf_ds = tf->tf_es = tf->tf_ss = USER_DS;
    tf->tf_esp = stacktop;// 栈顶位置为先前计算过的栈顶位置，注意在C语言的函数调用规范中，栈顶指针指向的位置应该是返回地址而不是第一个参数，这里让栈顶指针指向了第一个参数的原因在于，在中断返回之后，会跳转到ELF可执行程序的入口处，在该入口处会进一步使用call命令调用主函数，这时候也就完成了将 Return address 入栈的功能，因此这里无需画蛇添足压入返回地址
    tf->tf_eip = elf->e_entry;// 将返回地址设置为用户程序的入口
    tf->tf_eflags = FL_IF;// 允许中断，根据 IA32 的规范，eflags 的第 1 位需要恒为 1
    ret = 0;
```

```
    //(8)错误处理部分
out:
    return ret;             //返回
bad_cleanup_mmap:
    exit_mmap(mm);
bad_elf_cleanup_pgdir:
    put_pgdir(mm);
bad_pgdir_cleanup_mm:
    mm_destroy(mm);
bad_mm:
    goto out;
}
```

load_icode 主要是将文件加载到内存中执行，一共分为8个步骤：

- 1、建立内存管理器
- 2、建立页目录
- 3、将文件逐个段加载到内存中，这里要注意设置虚拟地址与物理地址之间的映射
- 4、建立相应的虚拟内存映射表
- 5、建立并初始化用户堆栈
- 6、处理用户栈中传入的参数
- 7、最后很关键的一步是设置用户进程的中断帧
- 8、发生错误还需要进行错误处理。

当然一旦发生错误还需要进行错误处理。

4、基于”UNIX的硬链接和软链接机制“的概要设方案

观察到保存在磁盘上的 inode 信息均存在一个 nlinks 变量用于表示当前文件的被链接的计数，因而支持实现硬链接和软链接机制；

★硬链接与软链接概念区别总结

硬链接

- 一般情况下，文件名和inode号码是"一一对应"关系，每个inode号码对应一个文件名。但是，Unix/Linux系统允许，多个文件名指向同一个inode号码。
- 这意味着，可以用不同的文件名访问同样的内容；对文件内容进行修改，会影响到所有文件名；但是，删除一个文件名，不影响另一个文件名的访问。这种情况就被

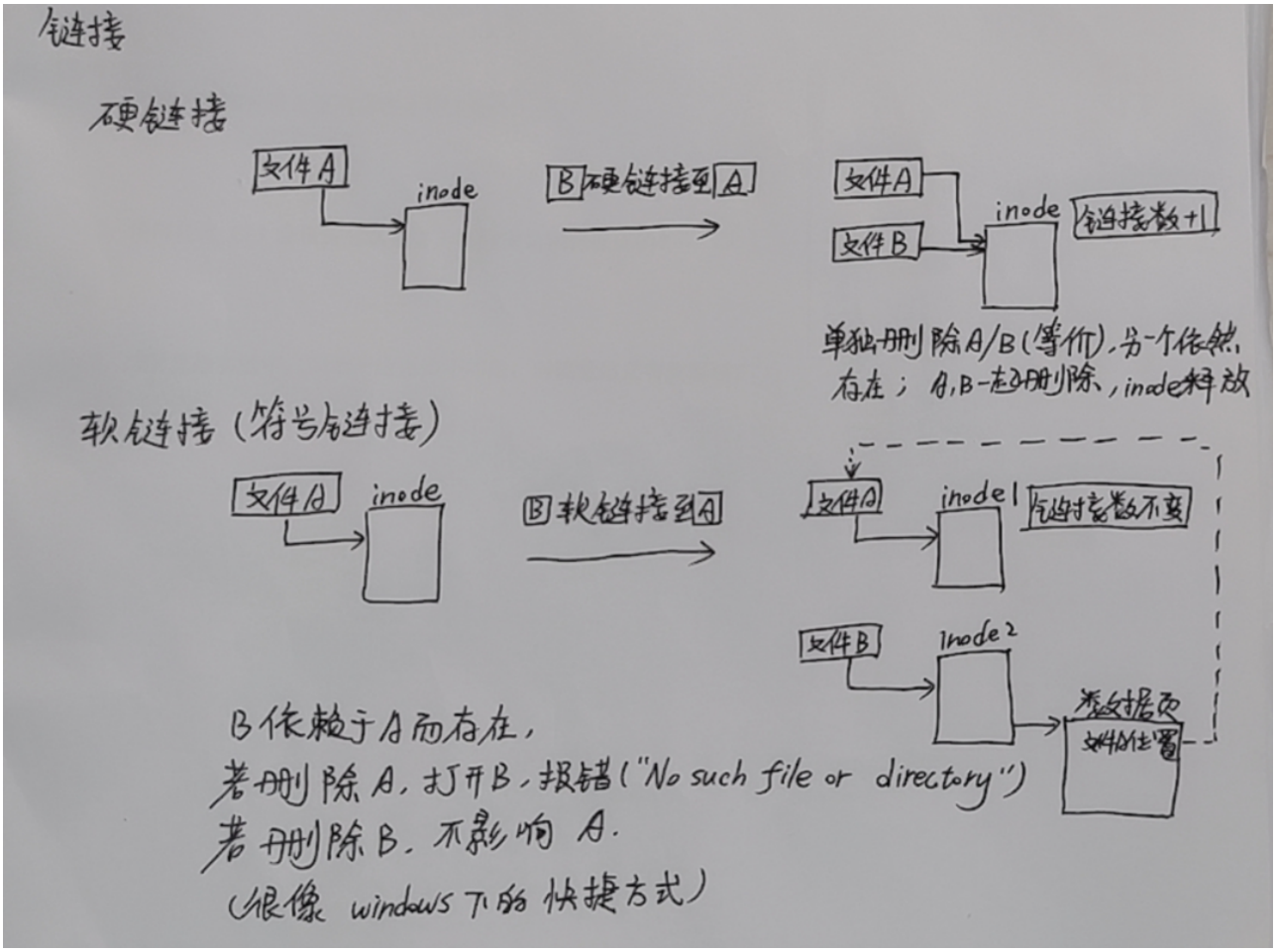
称为"硬链接"（hard link）。

- 共用同一个索引的许多不同名文件，修改一个文件会影响所有其他的文件，但是删除单个文件其他文件依然存在，只有在所有硬链接连接的文件都被删除之后，inode才会被释放。

软连接：

- 文件A和文件B的inode号码虽然不一样，但是文件B的内容是文件A的路径。读取文件B时，系统会自动将访问者导向文件A。因此，无论打开哪一个文件，最终读取的都是文件A。这时，文件B就称为文件A的"软链接"（soft link）或者"符号链接（symbolic link）。
- 这意味着，文件B依赖于文件A而存在，如果删除了文件A，打开文件B就会报错："No such file or directory"。这是软链接与硬链接最大的不同：文件B指向文件A的文件名，而不是文件A的inode号码，文件A的inode"链接数"不会因此发生变化。

总结图如下：



硬链接与软链接实现

软链接：

- 如果在磁盘上创建一个文件 A 的软链接 B，那么将 B 当成正常的文件创建 inode，然后将 TYPE 域设置为链接，然后使用剩余的域中的一个，指向 A 的 inode 位置，然后再额外使用一个位来标记当前的链接是软链接还是硬链接；
- 当访问到文件 B（read，write 等系统调用），判断如果 B 是一个链接，则实际是将对B指向的文件A（已经知道了 A 的 inode 位置）进行操作；
- 当删除一个软链接 B 的时候，直接将其在磁盘上的 inode 删掉即可；

硬链接：

- 如果在磁盘上的文件 A 创建一个硬链接 B，那么在按照软链接的方法创建完 B 之后，还需要将 A 中的被链接的计数加 1；
- 访问硬链接的方式与访问软链接是一致的；
- 当删除一个硬链接B的时候，除了需要删除掉 B 的 inode 之外，还需要将 B 指向的文件 A 的被链接计数减 1，如果减到了 0，则需要将 A 删除掉；

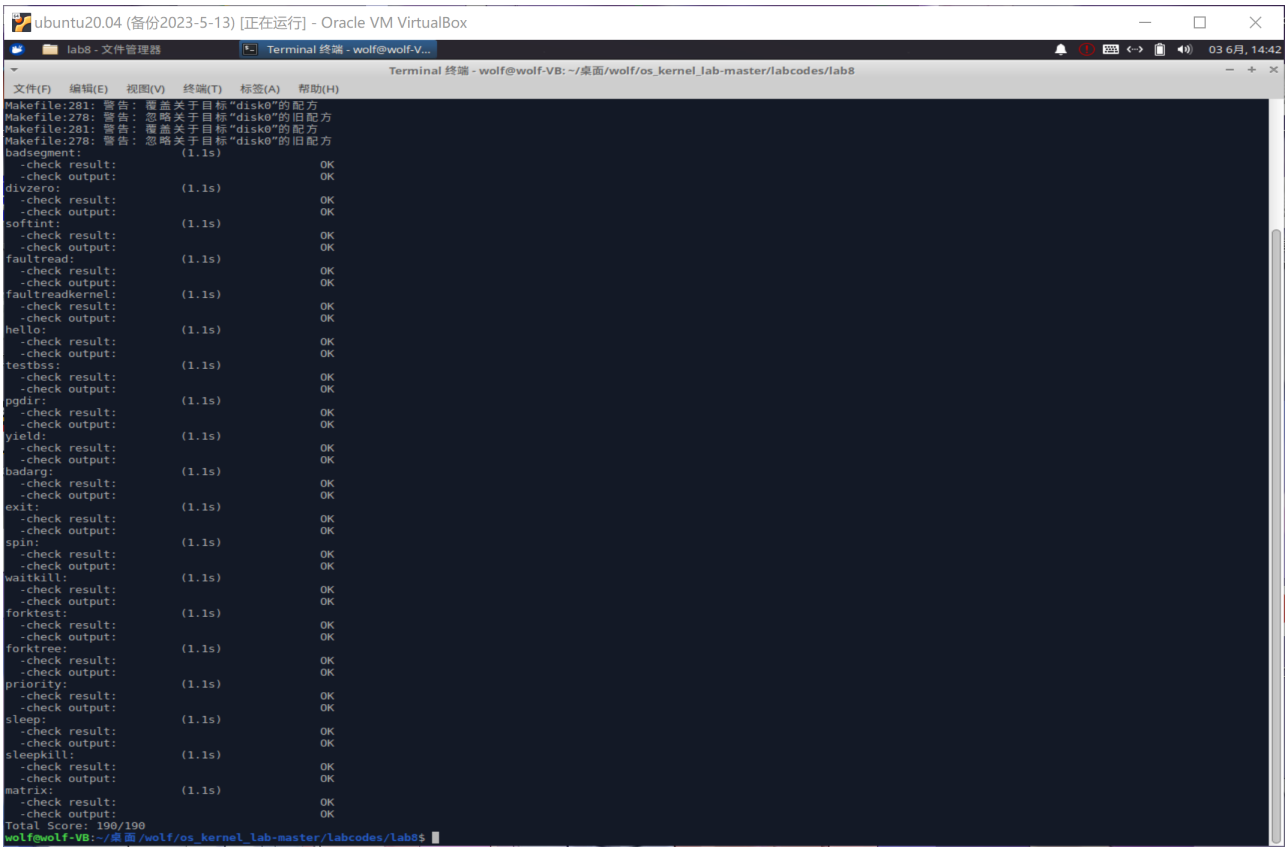
硬链接机制的实现：
①创建硬链接时，仍然为new_path建立一个sfs_disk_entry结构，但该结构的内部ino成员指向old_path的磁盘索引结点，并使该磁盘索引节点的nlinks引用计数成员加一即可。
②删除硬链接时，令对应磁盘结点sfs_disk_inode中的nlinks减一，同时删除硬链接的sfs_disk_entry结构即可。

软链接的实现：
①与创建硬链接不同，创建软链接时要多建立一个sfs_disk_inode结构（即建立一个全新的文件）。之后，将old_path写入该文件中，并标注sfs_disk_inode的type为SFS_TYPE_LINK即可。
②删除软链接与删除文件的操作没有区别，直接将对应的sfs_disk_entry和sfs_disk_inode结构删除即可。

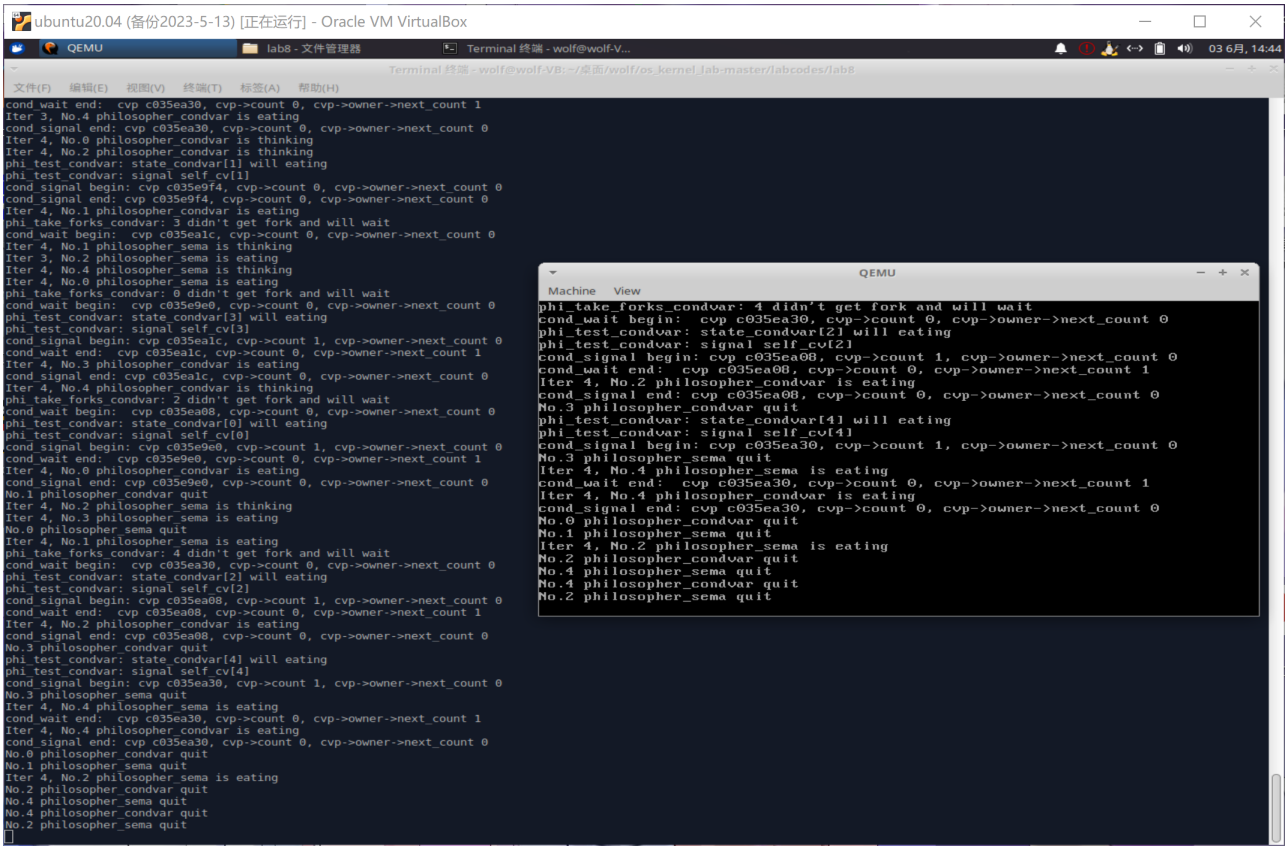
实验展示

注意：在make grade与make qemu之间一定要make clean，否则可能出错，推测是构建方式的不兼容。

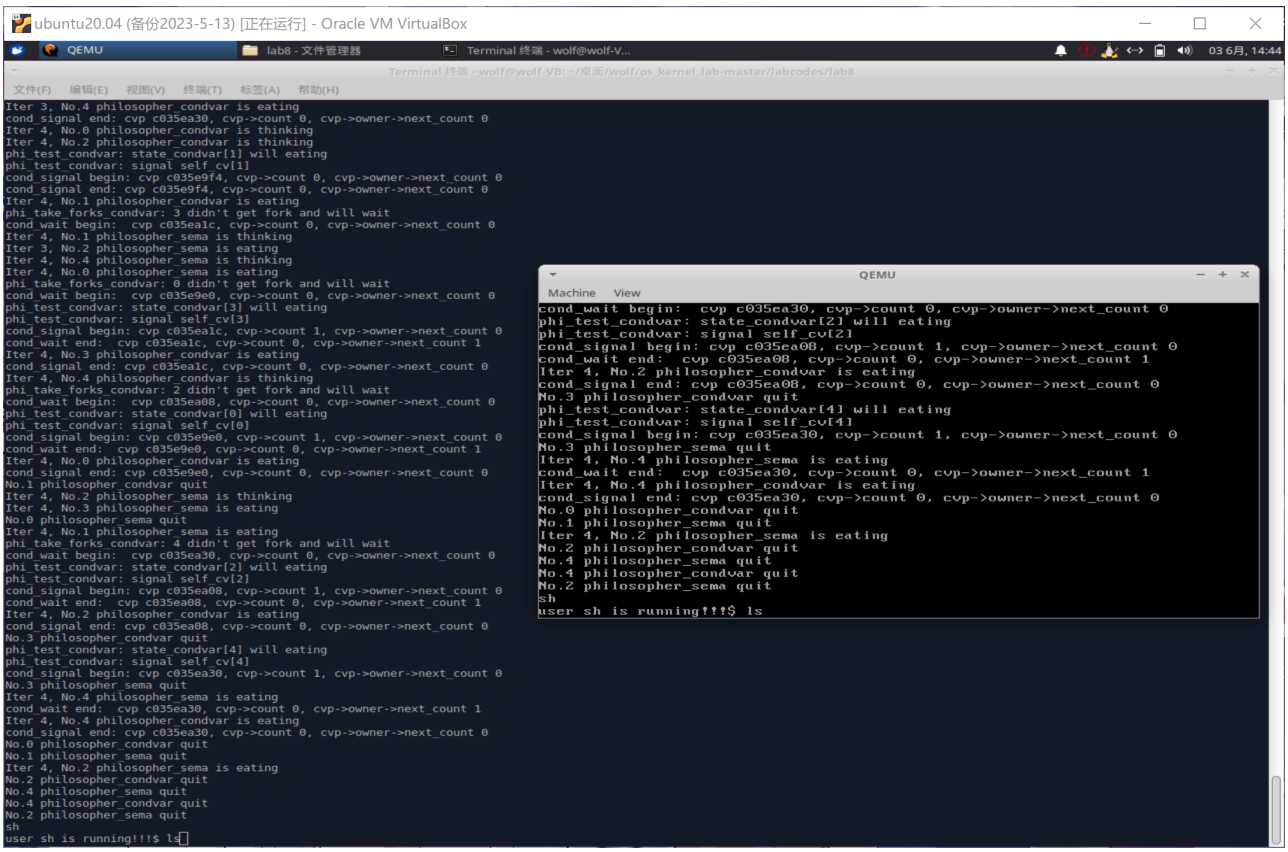
make grade



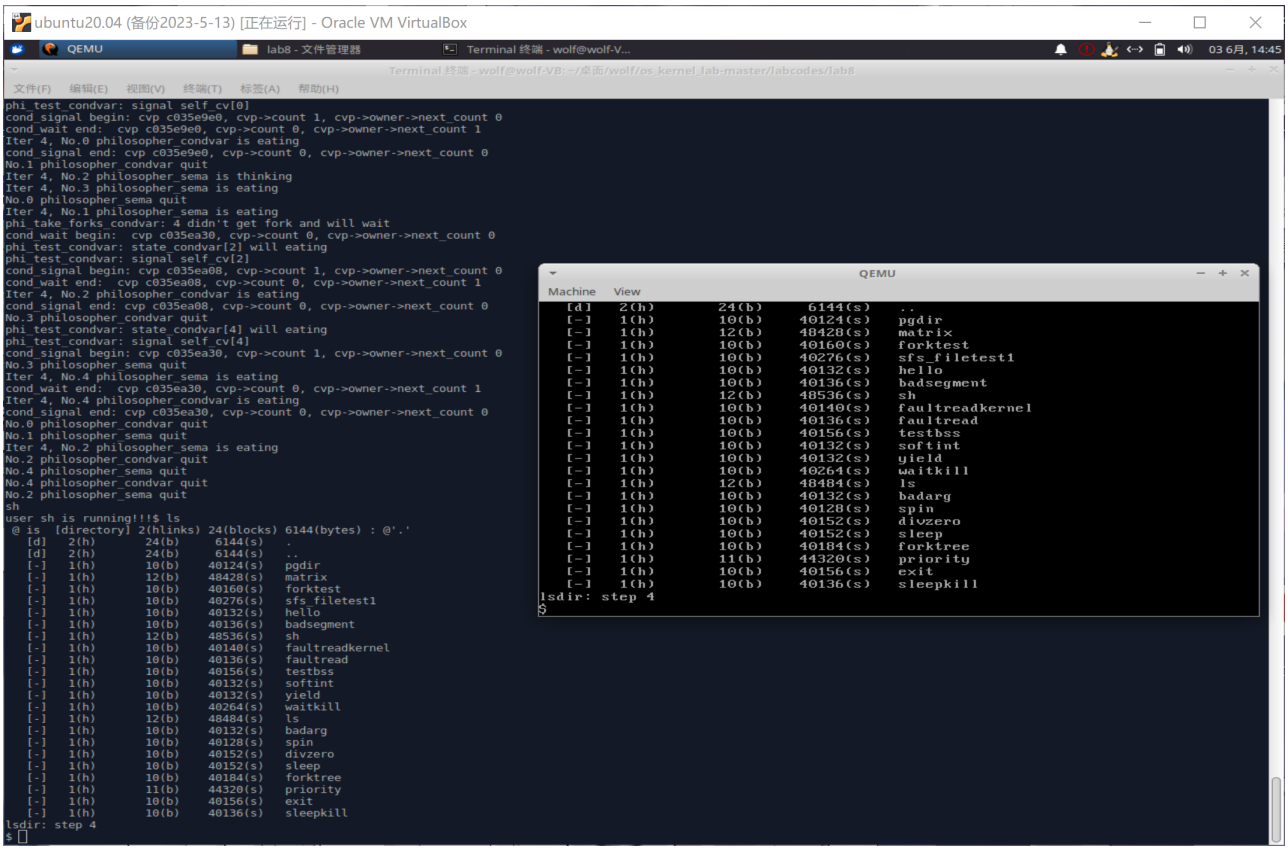
make qemu



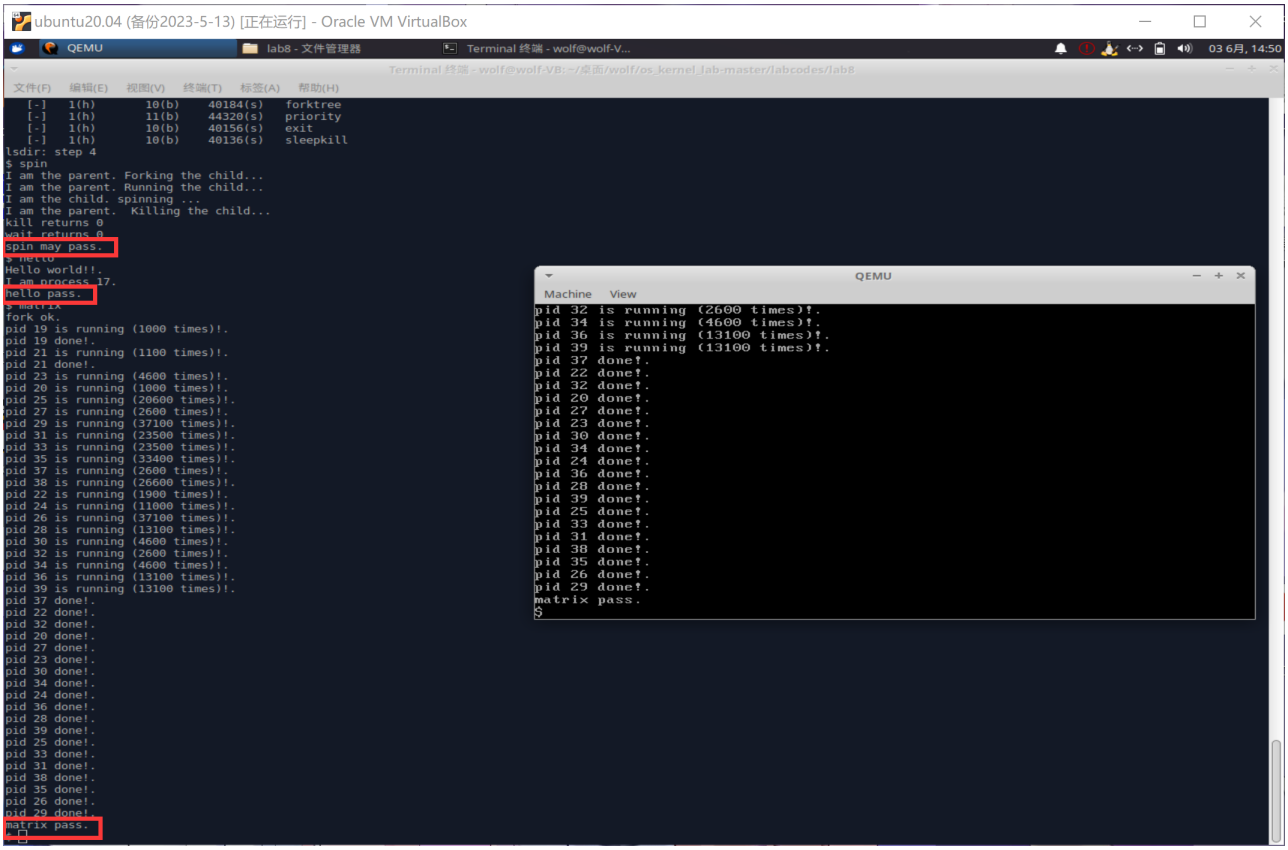
在信息输出完成之后，输入sh可以进入用户系统



在进入用户系统之后，我们输入ls，可以查看文件目录。



接下来我们可以输入文件目录中的文件，以查看文件。实际上ls本身也是一个文件，功能是显示文件列表，所以我们可以通过访问ls来看文件目录。



访问每一个文件，若成功访问，不仅会输出运行结果，在最后还会显示XXX pass，如“hello pass”等。

基本演示就到这里。到这个程度基本可以认为文件系统完成了。

实验总结

与实验七相比，实验八增加了文件系统，并因此实现了通过文件系统来加载可执行文件到内存中运行的功能，导致对进程管理相关的实现比较大的调整。首先是kern_init函数，可以发现与lab7相比增加了对fs_init函数的调用。fs_init函数就是文件系统初始化的总控函数，它进一步调用了虚拟文件系统初始化函数vfs_init，与文件相关的设备初始化函数dev_init和Simple FS文件系统的初始化函数sfs_init。这三个初始化函数联合在一起，协同完成了整个虚拟文件系统、SFS文件系统和文件系统对应的设备（键盘、串口、磁盘）的初始化工作。vfs_init主要建立了一个device list双向链表vdev_list，为后续具体设备（键盘、串口、磁盘）以文件的形式呈现建立查找访问通道。dev_init函数通过进一步调用disk0/stdin/stdout_device_init完成对具体设备的初始化，把它们抽象成一个设备文件，并建立对应的inode数据结构，最后把它们链入到vdev_list中。这样通过虚拟文件系统就可以方便地以文件的形式访问这些设备了。sfs_init是完成对Simple FS的初始化工作，并把此实例文件系统挂在虚拟文件系统中，从而让ucore的其他部分能够通过访问虚拟文件系统的接口来进一步访问到SFS实例文件系统。

相关知识点

- 文件系统架构
- SFS文件系统实现
- PIPE机制
- 基于文件系统的程序加载
- 硬链接与软链接机制

未涉及的知识点

本实验为ucore最后一个实验，实现了ucore的文件系统。至此，实验已经涉及了ucore的内存空间管理(分段与分页机制，进程虚拟内存空间管理)，进程管理及调度(包括进程调度策略和过程，上下文切换，系统调用等)，并发性问题(信号量，条件变量及管程)，文件系统的内容。已经基本涉及了操作系统的所有重要内容及知识点。

与参考答案的比较

练习1

与参考答案一致。主要是理解ucore中的文件系统构成，以及了解SFS文件系统。认识从通用文件系统访问接口层到文件系统抽象层，再到具体文件系统，最终到外设接口层调用实现文件访问的过程。对于pipe机制实现的思考题，由于对文件系统还有一部分理解不足的地方，只能简单给出需要完成的基本系统调用。

练习2

与参考答案一致。练习2主要是对已有实验中对文件访问相关的部分的修改，最核心的是加载用户程序的load_icode的修改，将其从内存中加载程序的部分改为从文件中读取程序内容。由于对文件系统还有一部分理解不足的地方，只能简单地按照注释提示给出代码的翻译。

参考资料

- 主要参考：https://github.com/AngelKitty/review_the_national_post-graduate_entrance_examination/blob/master/books_and_notes/professional_courses/operating_system/sources/ucore_os_lab/docs/lab_report/lab8/lab8%20%E5%AE%9E%E9%AA%8C%E6%8A%A5%E5%91%8A.md
- 练习一读函数部分：https://blog.csdn.net/sfadjlha/article/details/125352113
- 硬链接与软连接概念：https://www.likecs.com/show-308093557.html
- 硬链接与软链接实现：https://blog.csdn.net/CNRalap/article/details/129758090

- 实验总结参考：https://blog.csdn.net/CNRalap/article/details/129758090